Customer Segmentation (Clustering)

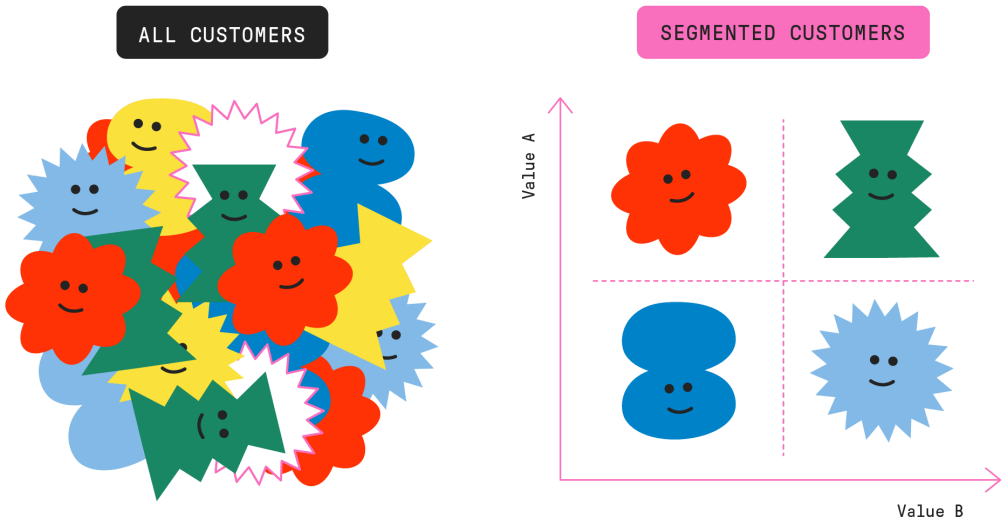


Table of Contents

- 1. Problem statement
  - 1.1. Introduction
  - 1.2. Dataset Features
- 2. Import Libraries and Data
- 3. Handling Missing Values
- 4. Feature Engineering
- 5. Data Analysis and Visualization
- 6. Outlier Detection
- 7. Check for Rare Categories
- 8. Feature Scaling
- 9. Dimensionality Reduction
- 10. Clustering
- 11. Clusters Analysis and Customers Profiling

1. Problem Statement

1.1. Introduction

Customer Personality Analysis is a detailed analysis of a company's ideal customers. It helps a business to better understand its customers and makes it easier for them to modify products according to the specific needs, behaviors and concerns of different types of customers.

Customer personality analysis helps a business to modify its product based on its target customers from different types of customer segments. For example, instead of spending money to market a new product to every customer in the company's database, a company can analyze which customer segment is most likely to buy the product and then market the product only on that particular segment.

1.2. Dataset Features

People:

- `ID` : Customer's unique identifier
- `Year_Birth` : Customer's birth year
- `Education` : Customer's education level
- `Marital_Status` : Customer's marital status
- `Income` : Customer's yearly household income
- `Kidhome` : Number of children in customer's household
- `Teenhome` : Number of teenagers in customer's household
- `Dt_Customer` : Date of customer's enrollment with the company
- `Recency` : Number of days since customer's last purchase
- `Complain` : 1 if the customer complained in the last 2 years, 0 otherwise

Products:

- `MntWines` : Amount spent on wine in last 2 years
- `MntFruits` : Amount spent on fruits in last 2 years
- `MntMeatProducts` : Amount spent on meat in last 2 years
- `MntFishProducts` : Amount spent on fish in last 2 years
- `MntSweetProducts` : Amount spent on sweets in last 2 years
- `MntGoldProds` : Amount spent on gold in last 2 years

Promotion:

- `NumDealsPurchases` : Number of purchases made with a discount
- `AcceptedCmp1` : 1 if customer accepted the offer in the 1st campaign, 0 otherwise
- `AcceptedCmp2` : 1 if customer accepted the offer in the 2nd campaign, 0 otherwise
- `AcceptedCmp3` : 1 if customer accepted the offer in the 3rd campaign, 0 otherwise
- `AcceptedCmp4` : 1 if customer accepted the offer in the 4th campaign, 0 otherwise
- `AcceptedCmp5` : 1 if customer accepted the offer in the 5th campaign, 0 otherwise
- `Response` : 1 if customer accepted the offer in the last campaign, 0 otherwise

Place:

- `NumWebPurchases` : Number of purchases made through the company's website
- `NumCatalogPurchases` : Number of purchases made using a catalogue
- `NumStorePurchases` : Number of purchases made directly in stores

- `NumWebVisitsMonth` : Number of visits to company's website in the last month

2. Import Libraries and Data

In [1]:
```
# handle table-like data and matrices
import pandas as pd
import numpy as np

# visualisation
import seaborn as sns
import matplotlib.pyplot as plt
import missingno as msno
import plotly.express as px
import plotly.graph_objects as go
from plotly.subplots import make_subplots
import plotly.figure_factory as ff

# preprocessing
from sklearn.preprocessing import StandardScaler

# pca
from sklearn.decomposition import PCA

# clustering
from yellowbrick.cluster import KElbowVisualizer
from sklearn.cluster import KMeans, AgglomerativeClustering

# evaluations
from sklearn.metrics import confusion_matrix

# ignore warnings
import warnings
warnings.filterwarnings('ignore')

# to display the total number columns present in the dataset
pd.set_option('display.max_columns', None)
```

In [2]:
```
data = pd.read_csv('marketing_campaign.csv', sep="\t")
```

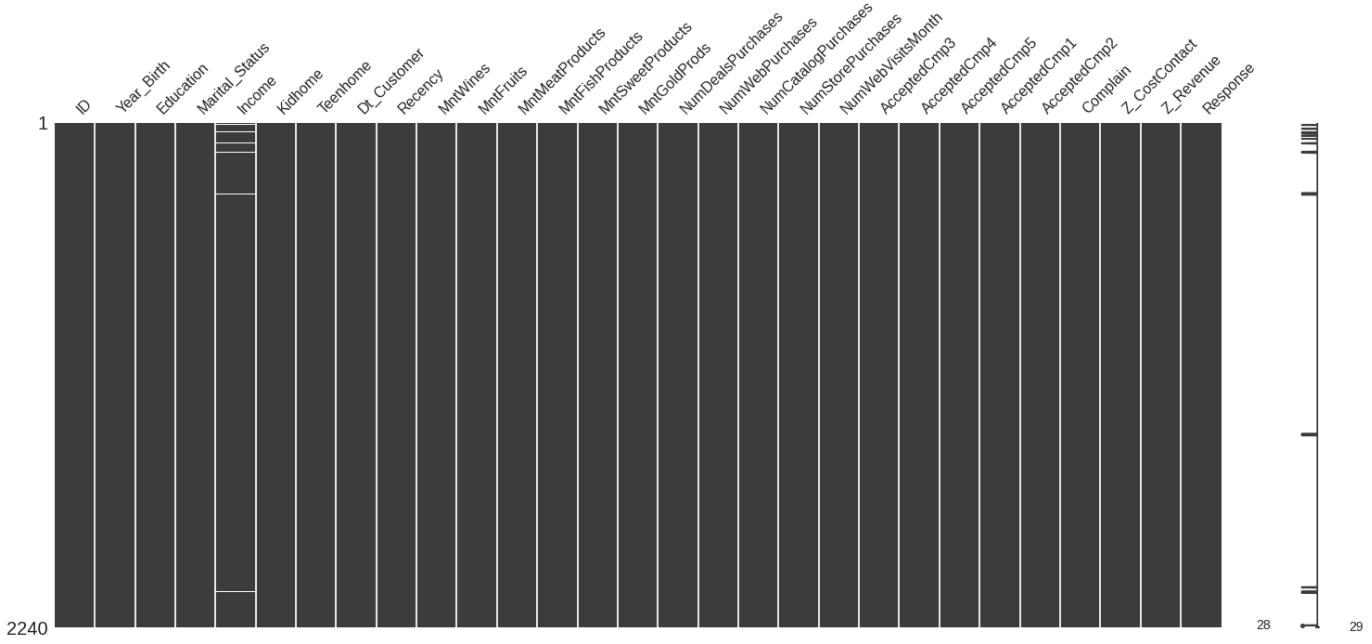
3. Handling Missing Values

let's find if we have missing values in the dataset.

In [3]:
```
data.isnull().sum()
```

Out[3]:
```
ID                     0
Year_Birth             0
Education              0
Marital_Status         0
Income                24
Kidhome                0
Teenhome               0
Dt_Customer            0
Recency                0
MntWines               0
MntFruits              0
MntMeatProducts        0
MntFishProducts        0
MntSweetProducts       0
MntGoldProds           0
NumDealsPurchases      0
NumWebPurchases        0
NumCatalogPurchases    0
NumStorePurchases      0
NumWebVisitsMonth      0
AcceptedCmp3           0
AcceptedCmp4           0
AcceptedCmp5           0
AcceptedCmp1           0
AcceptedCmp2           0
Complain               0
Z_CostContact          0
Z_Revenue              0
Response               0
dtype: int64
```

In [4]:
```
msno.matrix(data);
```



In [5]:
```
data = data.dropna()
```

In [6]:
```
data.isnull().sum()
```

Out[6]:
```
ID                   0
Year_Birth           0
Education            0
Marital_Status       0
Income               0
Kidhome              0
Teenhome             0
Dt_Customer          0
Recency              0
MntWines             0
MntFruits            0
MntMeatProducts      0
MntFishProducts      0
MntSweetProducts     0
MntGoldProds         0
NumDealsPurchases    0
NumWebPurchases      0
NumCatalogPurchases  0
NumStorePurchases    0
NumWebVisitsMonth    0
AcceptedCmp3         0
AcceptedCmp4         0
AcceptedCmp5         0
AcceptedCmp1         0
AcceptedCmp2         0
Complain             0
Z_CostContact        0
Z_Revenue            0
Response             0
dtype: int64
```

let's find if we have duplicate rows.

In [7]:
```
data.duplicated().sum()
```

Out[7]:
```
0
```

4. Feature Engineering

In [8]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2216 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   ID                   2216 non-null   int64  
 1   Year_Birth           2216 non-null   int64  
 2   Education            2216 non-null   object 
 3   Marital_Status       2216 non-null   object 
 4   Income               2216 non-null   float64
 5   Kidhome              2216 non-null   int64  
 6   Teenhome             2216 non-null   int64  
 7   Dt_Customer          2216 non-null   object 
 8   Recency              2216 non-null   int64  
 9   MntWines             2216 non-null   int64  
 10  MntFruits            2216 non-null   int64  
 11  MntMeatProducts      2216 non-null   int64  
 12  MntFishProducts      2216 non-null   int64  
 13  MntSweetProducts     2216 non-null   int64  
 14  MntGoldProds         2216 non-null   int64  
 15  NumDealsPurchases    2216 non-null   int64  
 16  NumWebPurchases      2216 non-null   int64  
 17  NumCatalogPurchases  2216 non-null   int64  
 18  NumStorePurchases    2216 non-null   int64  
 19  NumWebVisitsMonth    2216 non-null   int64  
 20  AcceptedCmp3         2216 non-null   int64  
 21  AcceptedCmp4         2216 non-null   int64  
 22  AcceptedCmp5         2216 non-null   int64  
 23  AcceptedCmp1         2216 non-null   int64  
 24  AcceptedCmp2         2216 non-null   int64  
 25  Complain             2216 non-null   int64  
 26  Z_CostContact        2216 non-null   int64  
 27  Z_Revenue            2216 non-null   int64  
 28  Response             2216 non-null   int64  
dtypes: float64(1), int64(25), object(3)
memory usage: 519.4+ KB
```

Dt_Customer that indicates the date a customer joined the database is not parsed as DateTime

In [9]:
```
data['Dt_Customer'] = pd.to_datetime(data['Dt_Customer'])
```

In [10]:
```
print("The newest customer's enrolment date in the records:", max(data['Dt_Customer']))
print("The oldest customer's enrolment date in the records:", min(data['Dt_Customer']))
```

```
The newest customer's enrolment date in the records: 2014-12-06 00:00:00
The oldest customer's enrolment date in the records: 2012-01-08 00:00:00
```

Extract the "Age" of a customer by the "Year_Birth" indicating the birth year of the respective person.

In [11]:
```
data['Age'] = 2015 - data['Year_Birth']
```

Create another feature "Spent" indicating the total amount spent by the customer in various categories over the span of two years.

In [12]:
```
data['Spent'] = data['MntWines'] + data['MntFruits'] + data['MntMeatProducts'] + data['MntFis
```

Create another feature "Living_With" out of "Marital_Status" to extract the living situation of couples.

In [13]:
```
data['Living_With'] = data['Marital_Status'].replace({'Married':'Partner', 'Together':'Partne
```

Create a feature "Children" to indicate total children in a household that is, kids and teenagers.

In [14]:
```
data['Children'] = data['Kidhome'] + data['Teenhome']
```

To get further clarity of household, Creating feature indicating "Family_Size"

In [15]:
```
data['Family_Size'] = data['Living_With'].replace({'Alone': 1, 'Partner':2}) + data['Children
```

Create a feature "Is_Parent" to indicate parenthood status

In [16]:
```
data['Is_Parent'] = np.where(data.Children > 0, 1, 0)
```

Segmenting education levels in three groups

In [17]:
```
data['Education'] = data['Education'].replace({'Basic':'Undergraduate', '2n Cycle':'Undergrad
```

Dropping some of the redundant features

In [18]:
```
to_drop = ['Marital_Status', 'Dt_Customer', 'Z_CostContact', 'Z_Revenue', 'Year_Birth', 'ID']
data = data.drop(to_drop, axis=1)
```

In [19]:
```
data.head(3)
```

Out[19]:

	Education	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProducts	MntFi
0	Graduate	58138.0	0	0	58	635	88	546	
1	Graduate	46344.0	1	1	38	11	1	6	
2	Graduate	71613.0	0	0	26	426	49	127	

5. Data Analysis and Visualization

In [20]:
```
data.shape
```

Out[20]:
```
(2216, 29)
```

There are 2216 cutomers and 29 features in the dataset.

In [21]:
```
data.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2216 entries, 0 to 2239
Data columns (total 29 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   Education            2216 non-null   object
 1   Income               2216 non-null   float64
 2   Kidhome              2216 non-null   int64
 3   Teenhome             2216 non-null   int64
 4   Recency              2216 non-null   int64
 5   MntWines             2216 non-null   int64
 6   MntFruits            2216 non-null   int64
 7   MntMeatProducts      2216 non-null   int64
 8   MntFishProducts      2216 non-null   int64
 9   MntSweetProducts     2216 non-null   int64
 10  MntGoldProds         2216 non-null   int64
 11  NumDealsPurchases    2216 non-null   int64
 12  NumWebPurchases      2216 non-null   int64
 13  NumCatalogPurchases  2216 non-null   int64
 14  NumStorePurchases    2216 non-null   int64
 15  NumWebVisitsMonth    2216 non-null   int64
 16  AcceptedCmp3         2216 non-null   int64
 17  AcceptedCmp4         2216 non-null   int64
 18  AcceptedCmp5         2216 non-null   int64
 19  AcceptedCmp1         2216 non-null   int64
 20  AcceptedCmp2         2216 non-null   int64
 21  Complain             2216 non-null   int64
 22  Response             2216 non-null   int64
 23  Age                  2216 non-null   int64
 24  Spent                2216 non-null   int64
 25  Living_With          2216 non-null   object
 26  Children             2216 non-null   int64
 27  Family_Size          2216 non-null   int64
 28  Is_Parent            2216 non-null   int64
dtypes: float64(1), int64(26), object(2)
memory usage: 519.4+ KB
```

In [22]:
```
data.describe()
```

Out[22]:

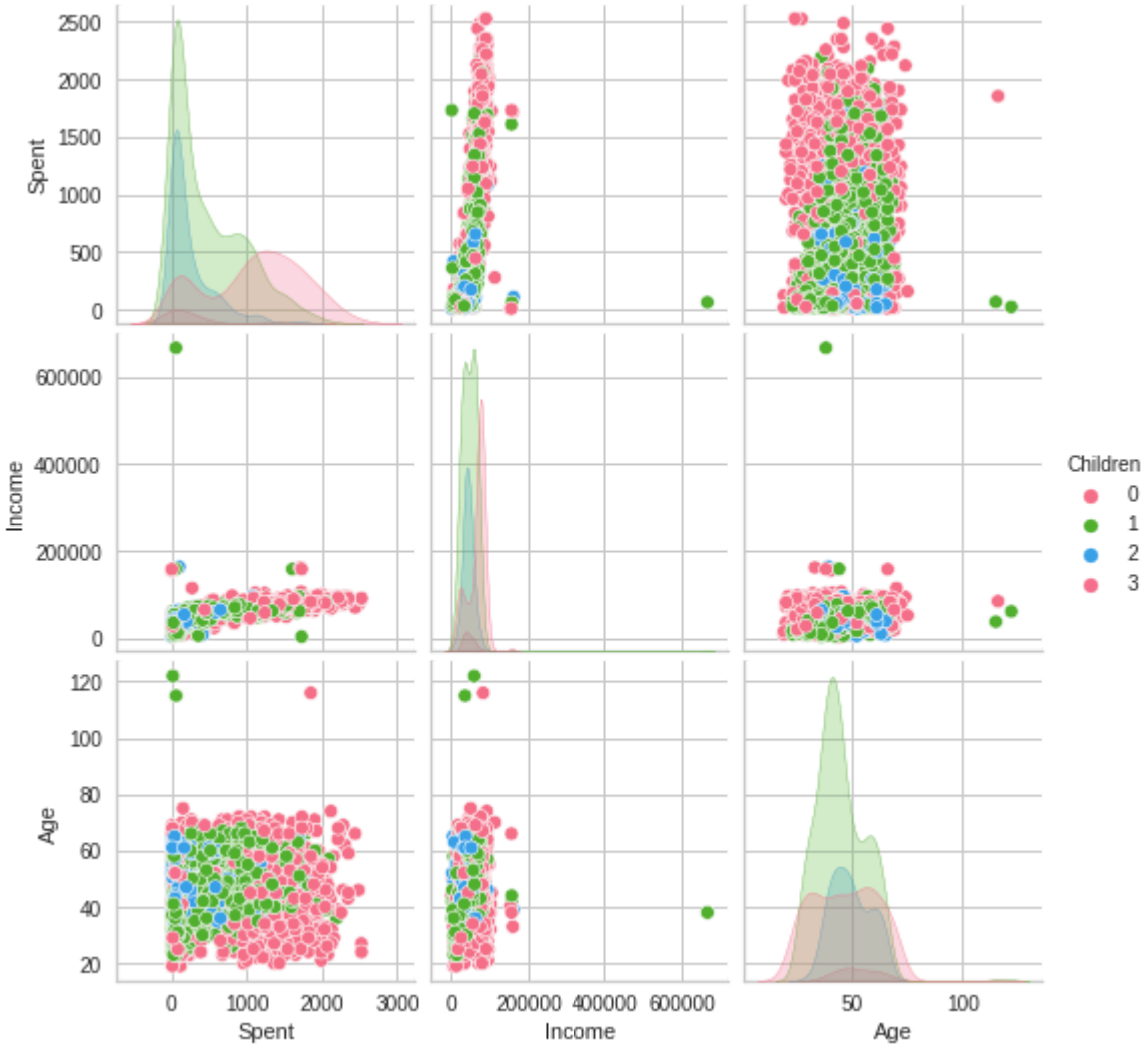
	Income	Kidhome	Teenhome	Recency	MntWines	MntFruits	MntMeatProdu
count	2216.000000	2216.000000	2216.000000	2216.000000	2216.000000	2216.000000	2216.000
mean	52247.251354	0.441787	0.505415	49.012635	305.091606	26.356047	166.995
std	25173.076661	0.536896	0.544181	28.948352	337.327920	39.793917	224.283
min	1730.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000
25%	35303.000000	0.000000	0.000000	24.000000	24.000000	2.000000	16.000
50%	51381.500000	0.000000	0.000000	49.000000	174.500000	8.000000	68.000
75%	68522.000000	1.000000	1.000000	74.000000	505.000000	33.000000	232.250
max	666666.000000	2.000000	2.000000	99.000000	1493.000000	199.000000	1725.000

In [23]:
```
data.describe(include=object).T
```

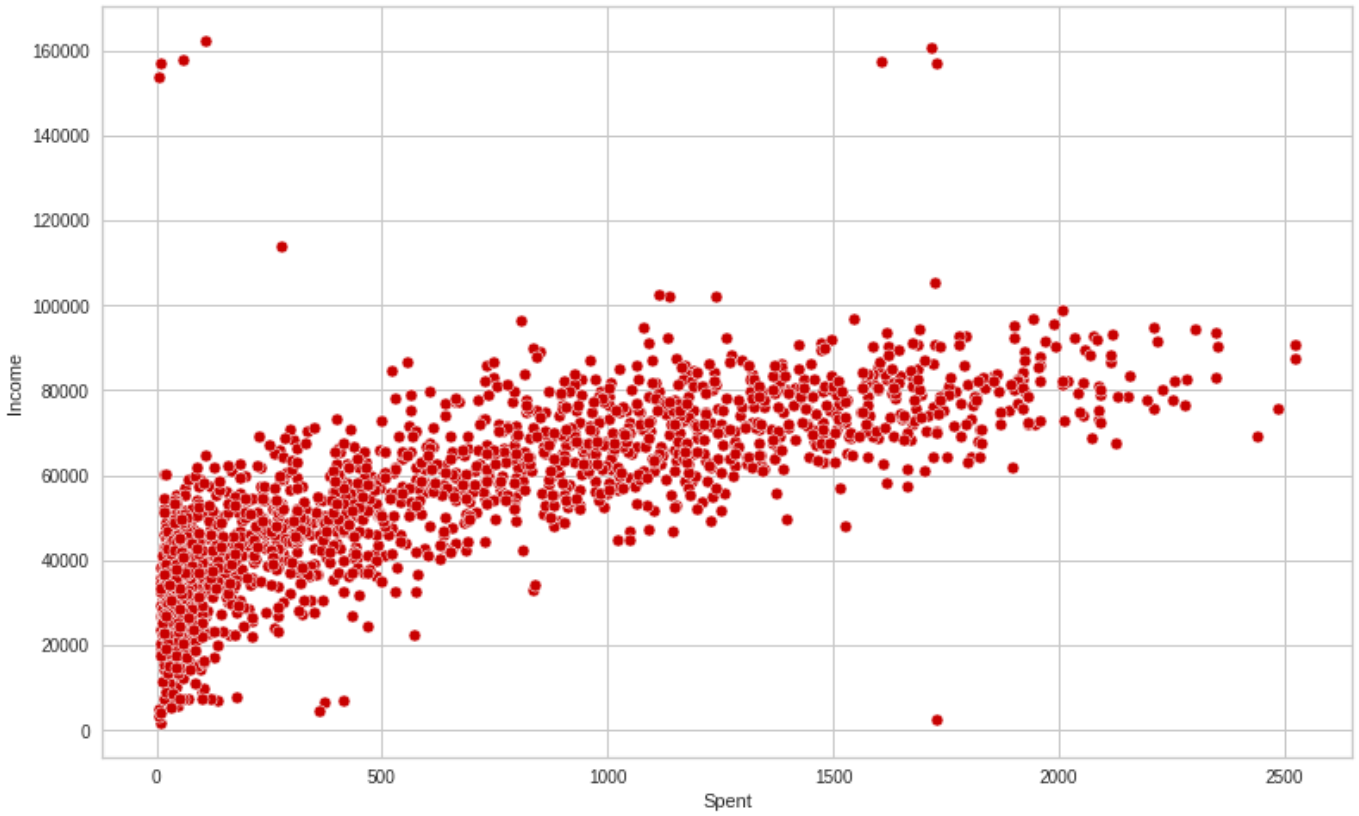
Out[23]:

	count	unique	top	freq
Education	2216	3	Graduate	1116
Living_With	2216	2	Partner	1430

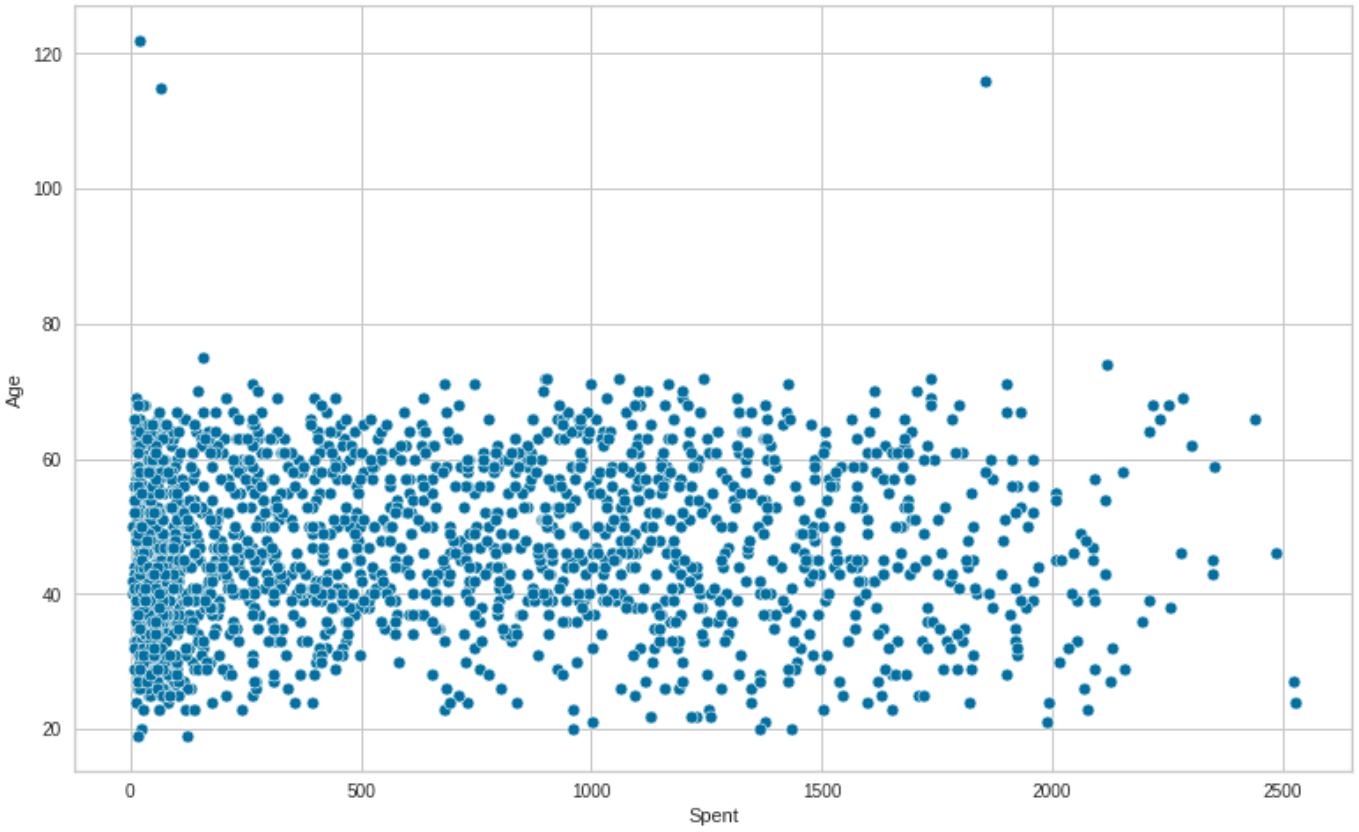
In [49]:
```
sns.pairplot(data , vars=['Spent','Income','Age'] , hue='Children', palette='husl');
```



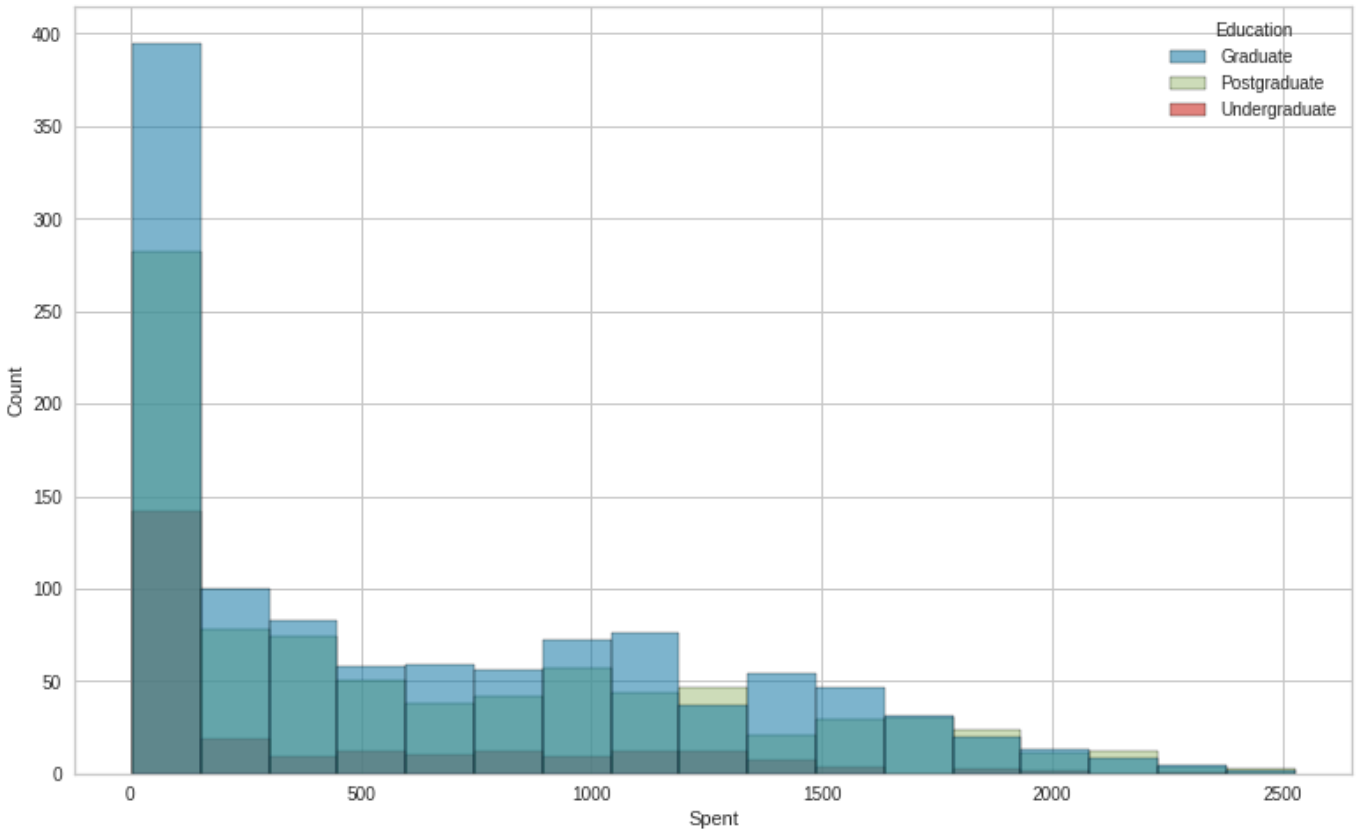
In [37]:
```
plt.figure(figsize=(13,8))
sns.scatterplot(x=data[data['Income']<600000]['Spent'], y=data[data['Income']<600000]['Income
```



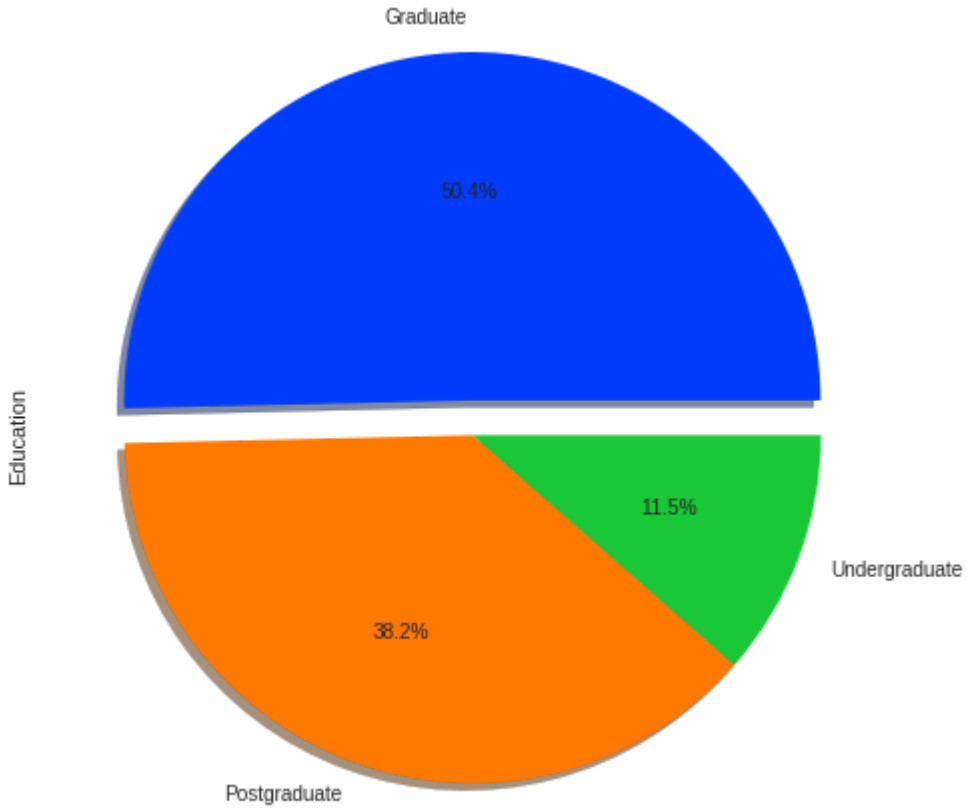
In [31]:
```
plt.figure(figsize=(13,8))
sns.scatterplot(x=data['Spent'], y=data['Age']);
```



In [26]:
```
plt.figure(figsize=(13,8))
sns.histplot(x=data['Spent'], hue=data['Education']);
```



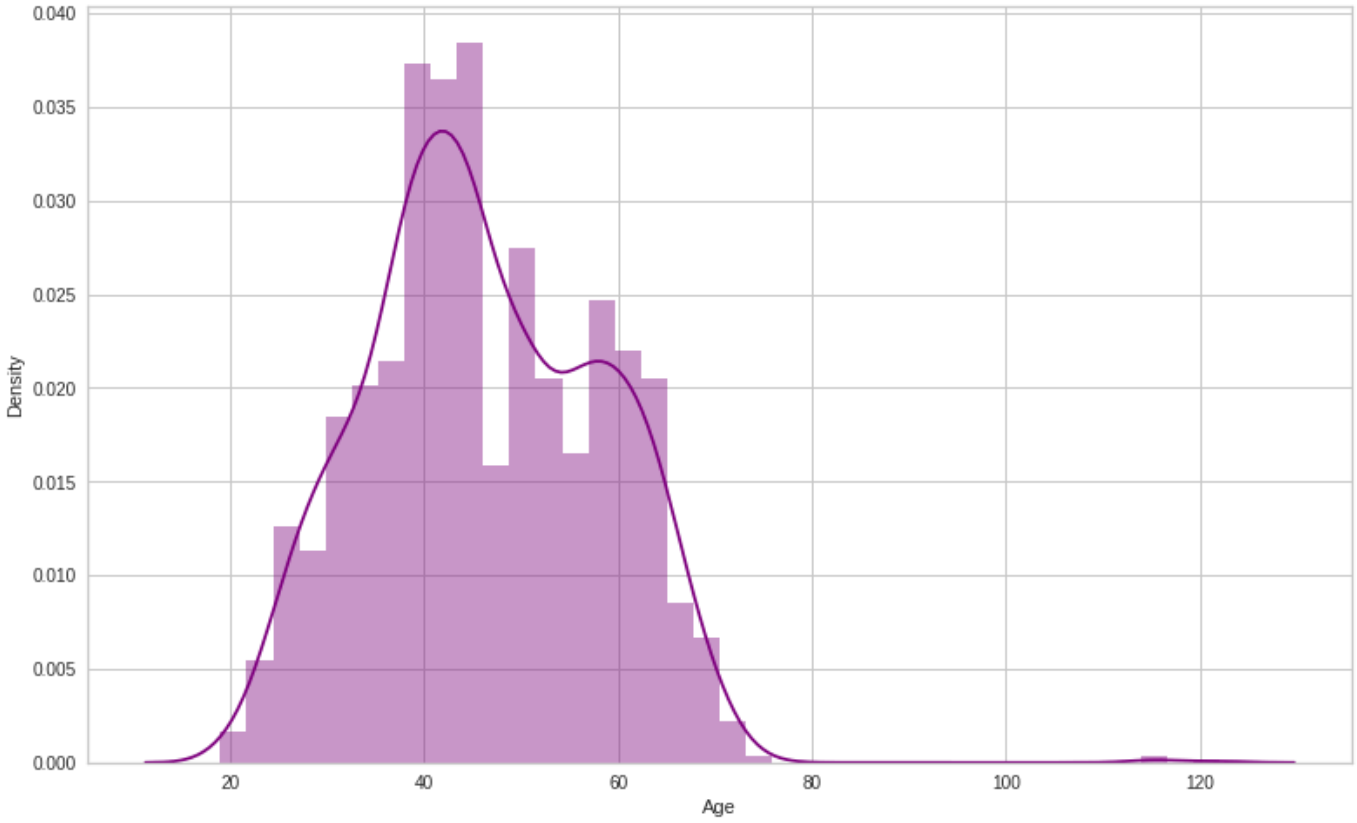
In [57]:
```
data['Education'].value_counts().plot.pie(explode=[0.1,0,0], autopct='%1.1f%%', shadow=True,
```



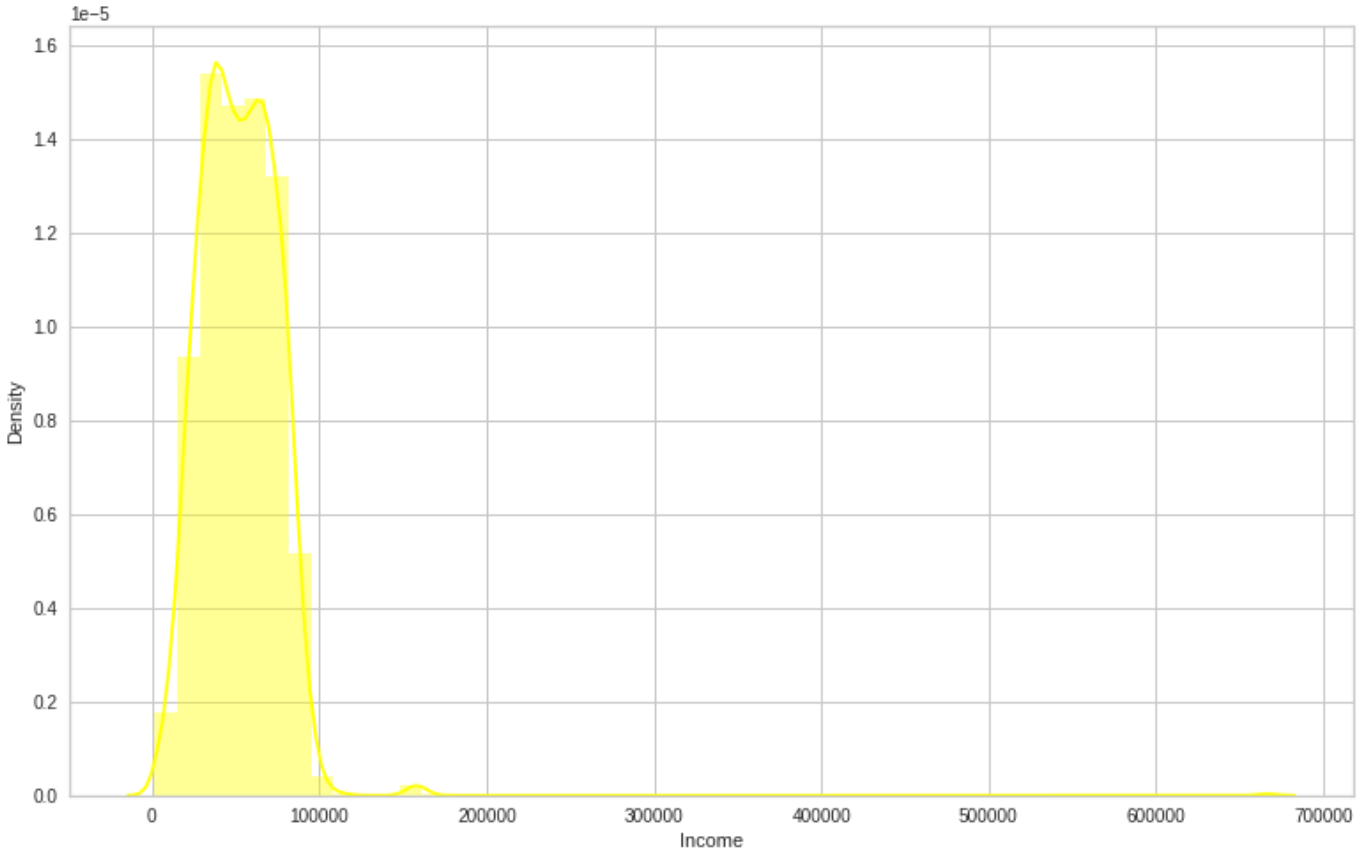
6. Outlier Detection

The presence of outliers in a classification or regression dataset can result in a poor fit and lower predictive modeling performance, therefore we should see there are ouliers in the data.

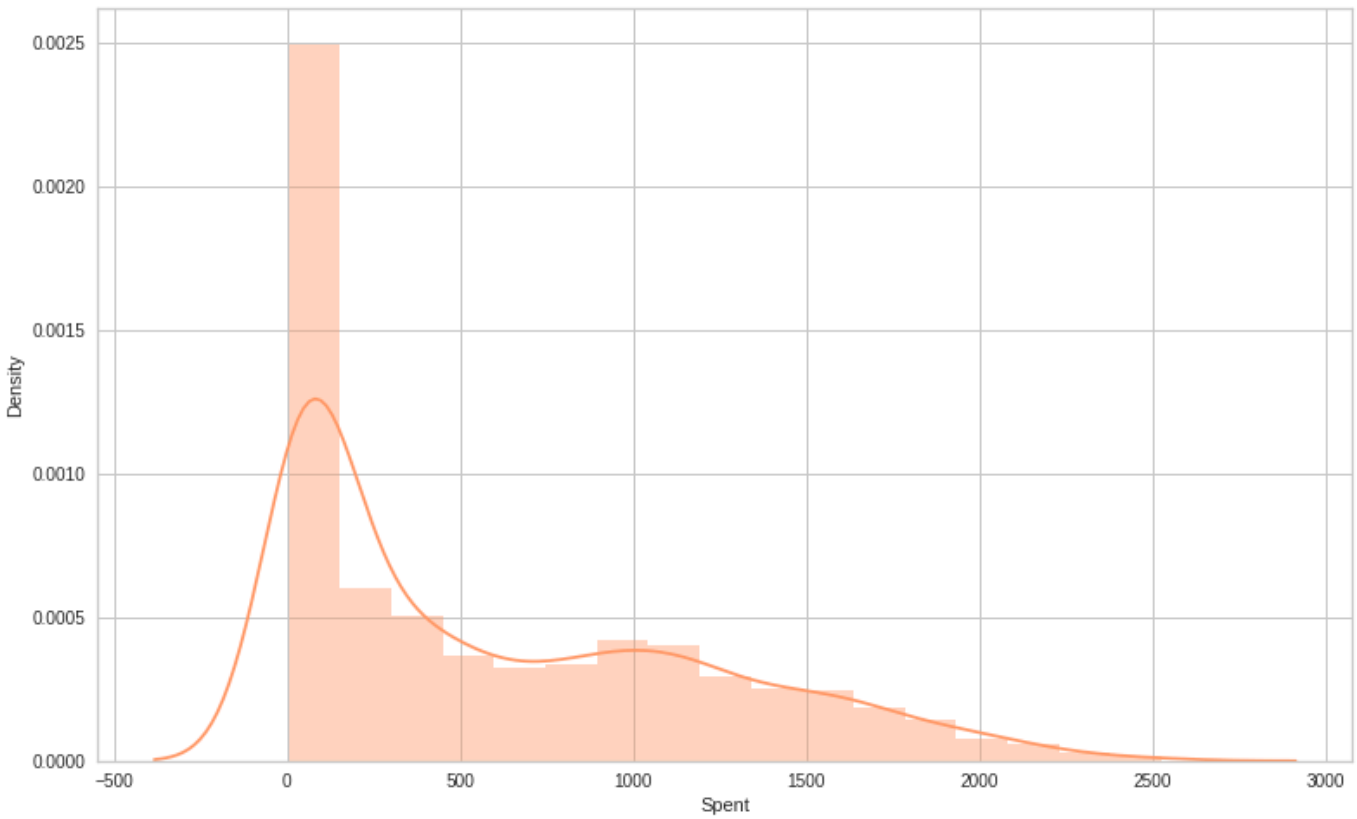
In [52]:
```
plt.figure(figsize=(13,8))
sns.distplot(data.Age, color='purple');
```



In [53]:
```
plt.figure(figsize=(13,8))
sns.distplot(data.Income, color='Yellow');
```



In [54]:
```
plt.figure(figsize=(13,8))
sns.distplot(data.Spent, color='#ff9966');
```



Another way of visualising outliers is using boxplots and whiskers, which provides the quantiles (box) and inter-quantile range (whiskers), with the outliers sitting outside the error bars (whiskers).

All the dots in the plot below are outliers according to the quantiles + 1.5 IQR rule

In [28]:
```
fig = make_subplots(rows=1, cols=3)

fig.add_trace(go.Box(y=data['Age'], notched=True, name='Age', marker_color = '#6699ff',
                     boxmean=True, boxpoints='suspectedoutliers'), 1, 2)

fig.add_trace(go.Box(y=data['Income'], notched=True, name='Income', marker_color = '#ff0066',
                     boxmean=True, boxpoints='suspectedoutliers'), 1, 1)

fig.add_trace(go.Box(y=data['Spent'], notched=True, name='Spent', marker_color = 'lightseagre
                     boxmean=True, boxpoints='suspectedoutliers'), 1, 3)

fig.update_layout(title_text='<b>Box Plots for Numerical Variables<b>')

fig.show()
```